

FORMATION PRATIQUE — 124 SLIDES

GitHub Copilot, en pratique

Transactis

Commandes, coûts, agents, MCP, skills, hooks, plugins, workflows. Français simple, zéro blabla, tout sourcé. Basé sur docs.github.com/copilot, code.visualstudio.com et github.com/github/awesome-copilot.



1 jour · pratique

Sources 100% citées

Exercice : TaskBoard (HTML/CSS uniquement)

hello@dhcompany.pro

FORMATEUR

Qui je suis

Joël Parfait Kuate

Ingénieur IA · Consultant & Formateur certifié

- Master en Architecture Logicielle & Big Data
- 7+ ans de formation — 600+ organisations formées en Belgique
- Fondateur : Digital House Company (Belgique), MS JOYCE SARL (Cameroun)
- Auteur : « L'Afrique et l'Intelligence Artificielle » (2024)
- Projets : ia4africa.org · FormIA · SANDYKIT · Jurisis.be · Indibot.be

hello@dhcompany.pro · @DigitalMaker237

3 documents, un seul dossier

1. Ce support (124 slides)

Théorie condensée, toutes les commandes, coûts, agents, MCP, skills, hooks, plugins, workflows. Chaque fait cite sa source.

2.

Exercice_Pratique_10_Etapes_3_Niveaux.md

10 étapes, chacune en 3 pistes : ● Facile, ● Intermédiaire, ● Pro.

3. codebase_taskboard/

Un site HTML/CSS uniquement (12 fichiers, aucun JavaScript) à manipuler avec Copilot.

? Avez-vous déjà utilisé Copilot sur un projet front-end (HTML/CSS) ?

SOURÇAGE

Convention : chaque info cite sa source

- Ligne **Source** sous chaque slide technique : documentation officielle exacte, consultée le 19/07/2026.
- 3 familles de sources : **GitHub Docs** (docs.github.com/copilot), **VS Code Docs** (code.visualstudio.com), **awesome-copilot** (github.com/github/awesome-copilot).
- 2 slides récapitulatives en fin de support listent toutes les URLs utilisées.

? Pourquoi vérifier la source d'une information technique plutôt que de la prendre telle quelle ?

PROGRAMME

Agenda de la journée

09h00-09h30	Fondamentaux	Panorama Copilot, gratuit vs payant
09h30-10h15	Coûts & sobriété	AI Credits, tokens, optimisation
10h15-11h15	Fiche de référence	Toutes les commandes, tous les modèles
11h15-12h15	Agents & écosystème avancé	Agent mode, cloud agent, MCP, skills, hooks, plugins, workflows
12h15-13h15	Pause déjeuner	—
13h15-14h00	Personnalisation (VS Code)	copilot-instructions.md, applyTo, AGENTS.md
14h00-14h45	Bien formuler sa demande	Prompt engineering officiel
14h45-15h00	Sécurité	Les règles Transactis
15h00-17h30	Pratique sur TaskBoard	Exercice en 10 étapes, 3 niveaux
17h30-17h45	Synthèse	Plan d'action

CULTURE IA

Comprendre l'IA avant d'utiliser Copilot

Définitions, types, niveaux, modèles, LLM, et comment on les compare — pour poser des bases solides avant la pratique.

DÉFINITION

Qu'est-ce que l'intelligence artificielle ?

L'intelligence artificielle (IA) désigne l'ensemble des techniques qui permettent à une machine de réaliser des tâches qui, jusqu'ici, demandaient une forme d'intelligence humaine : reconnaître une image, comprendre une phrase, jouer aux échecs, ou écrire une ligne de code. Ce n'est pas une seule technologie, mais une famille de méthodes très différentes, qui partagent un même objectif : produire un comportement « intelligent » à partir de données et de calcul, sans que chaque règle soit écrite à la main par un humain.

Il est important de garder une distinction claire en tête tout au long de cette formation : une IA **simule** un comportement intelligent en repérant des régularités statistiques dans d'énormes quantités de données. Elle ne « comprend » pas au sens où un humain comprend, elle ne possède ni conscience ni intentions. C'est cette nuance qui explique pourquoi Copilot peut produire une réponse qui a l'air juste, mais qui est fausse — et pourquoi la vérification humaine reste indispensable tout au long de cette journée.

? Avant cette formation, comment auriez-vous défini « l'intelligence artificielle » en une phrase ?

Systemes à règles et Machine Learning

La façon la plus courante de classer les IA est de regarder **comment elles apprennent à faire ce qu'elles font**. Les tout premiers systèmes, dits « à base de règles » (rule-based), suivaient uniquement des instructions écrites explicitement par un développeur : « si la température dépasse 30°, alors afficher une alerte ». Ils sont prévisibles, mais rigides : ils ne savent traiter que les cas prévus à l'avance.

Le **Machine Learning** (apprentissage automatique) change d'approche : au lieu d'écrire les règles, on montre à l'algorithme des milliers d'exemples (des données), et c'est lui qui « apprend » à repérer les régularités. Un modèle de Machine Learning entraîné à reconnaître des e-mails indésirables n'a jamais reçu la règle « si le mot GRATUIT apparaît 5 fois » — il l'a déduite lui-même à partir d'exemples réels.

? Dans votre travail quotidien, utilisez-vous déjà un outil basé sur des règles fixes (ex. un correcteur orthographique simple) ?

Deep Learning et IA générative

Le **Deep Learning** (apprentissage profond) est une famille particulière de Machine Learning, basée sur des « réseaux de neurones artificiels » empilés en plusieurs couches — d'où le mot « profond ». C'est cette technique qui a rendu possibles les grandes avancées récentes : reconnaissance vocale, traduction automatique, génération d'images, et les modèles de langage comme ceux qui alimentent GitHub Copilot.

L'**IA générative** est une application du Deep Learning : au lieu de simplement classer ou reconnaître une donnée, elle en **produit une nouvelle** — un texte, une image, une ligne de code — qui n'existait pas telle quelle auparavant, mais qui est statistiquement cohérente avec tout ce que le modèle a appris. C'est exactement ce que fait Copilot lorsqu'il propose une fonction que vous n'avez jamais écrite : il ne la « copie » pas, il la génère token par token, comme le module suivant va l'expliquer.

Source : classification usuelle de l'IA par technique (systèmes à règles, Machine Learning, Deep Learning, IA générative), reprise notamment par IBM et Google Cloud dans leurs ressources pédagogiques grand public.

IA étroite, IA générale, super-IA

Une seconde classification, complémentaire à la première, distingue les IA non plus par leur technique, mais par l'**étendue de leurs capacités**. On parle alors de 3 niveaux. L'**IA étroite** (Narrow AI, ou ANI) est excellente sur une tâche précise et uniquement celle-ci : un système d'échecs ne sait pas conduire une voiture, un correcteur de code ne sait pas composer de la musique. **Toutes les IA existantes aujourd'hui, y compris GitHub Copilot et les modèles les plus avancés, sont des IA étroites** — même si elles semblent polyvalentes, elles restent spécialisées dans le traitement de texte, d'image ou de code.

L'**IA générale** (AGI, Artificial General Intelligence) désigne une IA hypothétique capable d'apprendre et de raisonner sur n'importe quelle tâche intellectuelle, au niveau d'un humain — elle n'existe pas encore. La **super-IA** (ASI, Artificial Super Intelligence) irait au-delà des capacités humaines dans tous les domaines. Ces deux derniers niveaux relèvent aujourd'hui de la recherche et de la prospective, pas d'un outil que vous utiliserez cette semaine.

Retenez ceci pour la suite de la journée : Copilot est une IA étroite, très performante sur le code, mais sans compréhension générale du monde ni de votre contexte métier — d'où l'importance de la revue humaine.

? Cette distinction ANI/AGI/ASI change-t-elle votre façon de vous représenter Copilot ?

Qu'est-ce qu'un modèle d'IA ?

Un **modèle** est le résultat concret de l'entraînement d'une IA : un ensemble de paramètres numériques (souvent plusieurs milliards) qui encodent, sous forme de calculs mathématiques, tout ce que le système a « appris » à partir de ses données d'entraînement. Concrètement, un modèle est un fichier — très volumineux — que l'on peut interroger : on lui donne une entrée (une question, du code), il produit une sortie (une réponse, une suggestion), en quelques fractions de seconde.

Quand vous ouvrez le sélecteur de modèle dans Copilot Chat, vous choisissez littéralement **quel fichier de paramètres** va traiter votre demande. Deux modèles différents (par exemple GPT-5.4 et Claude Sonnet 5) ont été entraînés séparément, par des équipes différentes, sur des données et avec des méthodes différentes : ils donneront donc des réponses différentes à la même question, avec des forces et un coût différents — c'est tout l'enjeu du choix de modèle abordé plus loin dans ce support.

Qu'est-ce qu'un LLM (grand modèle de langage) ?

Un **LLM** (Large Language Model, « grand modèle de langage ») est un type particulier de modèle d'IA générative, spécialisé dans le traitement du texte — et, pour les modèles utilisés par Copilot, également du code source. Il est dit « large » (grand) parce qu'il est entraîné sur des quantités massives de texte (une bonne partie du contenu public d'internet, des livres, et énormément de code source) et parce qu'il compte des milliards de paramètres internes.

Le principe de fonctionnement, une fois entraîné, est étonnamment simple à décrire : un LLM ne fait, techniquement, qu'une seule chose — **prédire le prochain mot (ou fragment de mot, appelé « token »)** le plus probable, étant donné tout ce qui a été écrit avant. Il répète cette opération token après token, des milliers de fois par seconde, pour construire une réponse complète. Cette simplicité de principe explique à la fois sa puissance (il a vu tellement d'exemples qu'il « devine » remarquablement bien) et ses limites (il n'y a, à aucun moment, une vérification de véracité — seulement une continuation statistiquement plausible).

? Le fait qu'un LLM « prédise juste le mot suivant » change-t-il votre confiance dans ses réponses ?

Comment un LLM « apprend »

L'entraînement d'un LLM se déroule, très schématiquement, en deux grandes phases. La première, le **pré-entraînement**, consiste à faire lire au modèle d'énormes quantités de texte et de code, en lui faisant deviner, des milliards de fois, le mot suivant d'une phrase tronquée — puis en corrigeant légèrement ses paramètres internes à chaque erreur. C'est un processus purement statistique, sans compréhension consciente, mais qui, à cette échelle, produit des capacités de langage et de raisonnement remarquablement utiles.

La seconde phase, l'**alignement** (fine-tuning et apprentissage par retour humain), ajuste ensuite le modèle pour qu'il réponde de façon utile, sûre et conforme aux instructions — c'est ce qui transforme un modèle « qui complète du texte » en un assistant « qui répond à une question ». C'est cette seconde phase qui rend Copilot capable de comprendre une consigne comme `/explain` plutôt que de simplement continuer votre phrase.

Ce que cela veut dire pour vous : un LLM ne « sait » rien au sens humain — il génère la continuation la plus plausible. D'où la règle d'or déjà vue : toujours vérifier.

Arena.ai : comparer et classer les modèles

Arena.ai (anciennement connu sous le nom de LMArena / Chatbot Arena) est une plateforme publique, indépendante des éditeurs de modèles, qui permet de comparer les LLM entre eux de façon transparente. Son principe, appelé « Battle Mode », est simple : un même prompt est envoyé à deux modèles différents, sans que l'utilisateur sache lesquels ; il vote ensuite pour la meilleure réponse. Des milliers de votes de ce type, agrégés, produisent un **classement (leaderboard)** public, régulièrement mis à jour, qui reflète les préférences réelles des utilisateurs plutôt que les seules annonces marketing des éditeurs.

Ce classement est une ressource précieuse pour choisir un modèle par défaut d'équipe dans Copilot : plutôt que de se fier uniquement au nom le plus récent, on peut consulter comment un modèle se positionne réellement, y compris sur des catégories spécifiques comme la génération de code.

Point de vigilance signalé par la plateforme elle-même : vos échanges peuvent être transmis aux fournisseurs des modèles testés – n'y soumettez jamais de code ou de donnée sensible, exactement comme pour Copilot.

Source : arena.ai (anciennement lmarena.ai), consulté le 19/07/2026.

? Consulteriez-vous un classement communautaire avant de choisir un modèle par défaut pour votre équipe ?

TRANSITION

Pourquoi tout cela compte pour Copilot

Ces notions ne sont pas de la théorie détachée de la pratique : elles expliquent directement ce que vous allez faire cet après-midi sur TaskBoard. Quand vous choisirez un modèle dans le sélecteur, vous choisirez un LLM précis, entraîné différemment des autres, avec des forces différentes. Quand Copilot proposera un code, vous saurez qu'il s'agit d'une prédiction statistique plausible, pas d'une vérité vérifiée — d'où l'importance de chaque checklist de ce support. Et quand nous parlerons de coûts, vous comprendrez pourquoi un « grand » modèle de langage, plus lourd à faire tourner, coûte plus cher qu'un modèle « mini ».

? Une dernière question avant de continuer : quelle notion de ce module allez-vous réexpliquer à un collègue cette semaine ?

FONDAMENTAUX

Panorama GitHub Copilot

Ce que c'est, où on le trouve, ce qui est gratuit.

C'est quoi, GitHub Copilot ?

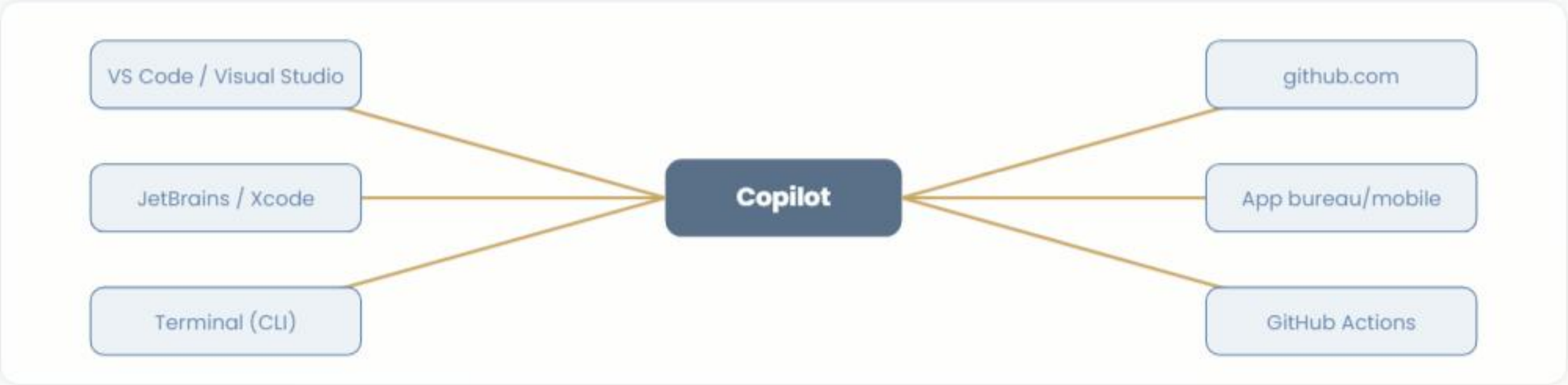
GitHub Copilot est un assistant IA construit autour de modèles de langage (les LLM que nous venons de définir), entraînés notamment sur du code source, et connecté directement à votre projet et à votre éditeur. Contrairement à un chat généraliste, il a accès à vos fichiers, à votre structure de projet et à l'historique de votre conversation, ce qui lui permet de répondre en tenant compte du contexte réel de votre travail plutôt que de généralités.

Le principe reste toutefois identique à celui de n'importe quel LLM : Copilot **propose**, vous **décidez**. Chaque suggestion, chaque ligne de code générée, doit être lue et vérifiée avant d'être acceptée — ce n'est pas une précaution excessive, c'est une conséquence directe de la façon dont un modèle de langage fonctionne, comme nous venons de le voir.

Source : *GitHub Docs*, « *What is GitHub Copilot?* » — docs.github.com/en/copilot/get-started/what-is-github-copilot (consulté le 19/07/2026).

? Qu'attendez-vous personnellement de Copilot sur vos projets front-end ?

Où trouve-t-on Copilot ?



Source : GitHub Docs, « Copilot agents concepts » — docs.github.com/en/copilot/concepts/agents (consulté le 19/07/2026).

? Dans quel environnement de cette liste travaillez-vous le plus ?

DEUX COPILOT

Copilot Chat (M365) vs GitHub Copilot

PRODUIT	POUR QUOI FAIRE	VOIT VOTRE CODE ?
Copilot Chat (M365)	Mail, résumé, Office	Non
GitHub Copilot	Code, tests, documentation, revue	Oui

Source : Microsoft Learn, « Decide which Copilot is right for you » (consulté le 08/07/2026).

Ce qui est gratuit, ce qui coûte

● Gratuit, illimité

Complétions de code et suggestions « next edit » pendant que vous tapez.

● Consomme des AI Credits

Chat, Agent mode, Cloud agent, Copilot CLI, Copilot Spaces, Spark.

Source : GitHub Docs, « Usage-based billing for individuals » — docs.github.com/en/copilot/concepts/billing/usage-based-billing-for-individuals (consulté le 19/07/2026).

[⚡ MINI-QUIZ ÉCLAIR](#)

Fondamentaux — 3 questions

1. Vrai ou faux : Copilot Chat M365 voit le code de vos dépôts.
2. Citez 2 environnements où GitHub Copilot est disponible, en dehors de VS Code.
3. Les complétions de code pendant que vous tapez sont-elles facturées ?

? On répond à voix haute, qui commence ?



COÛTS & SOBRIÉTÉ

Comment les coûts fonctionnent

AI Credits, tokens, et comment payer moins pour la même qualité.

C'est quoi, un token ?

Un **token** est l'unité de base que manipule un LLM comme ceux qui alimentent Copilot — vu au module précédent. C'est un petit morceau de texte, souvent un mot courant ou une partie de mot pour les termes plus rares ou le code (par exemple, `createTask` peut être découpé en plusieurs tokens). Le modèle ne « lit » et n'« écrit » jamais directement des phrases : il traite une suite de tokens, l'un après l'autre.

En pratique, une question de 20 mots représente environ 25 à 30 tokens d'entrée, tandis qu'une réponse contenant du code peut facilement atteindre plusieurs centaines de tokens de sortie. C'est cette unité — pas le nombre de « questions » posées — qui détermine le coût réel d'un échange avec Copilot, comme nous allons le voir avec le modèle des AI Credits.

? À votre avis, un fichier de 300 lignes collé dans le chat représente combien de tokens, à la louche ?

1 AI Credit = 0,01 \$

Toute consommation de Copilot Chat, Agent mode, Cloud agent ou CLI est convertie en **AI Credits**, l'unité de facturation à l'usage de Copilot depuis 2026. Chaque credit vaut exactement 0,01 \$, ce qui rend le coût d'une session directement traduisible en euros ou en dollars — un budget mensuel de 1 500 credits équivaut donc à 15 \$ de consommation possible.

Le coût final d'une demande dépend de deux facteurs combinés : le **modèle** choisi dans le sélecteur (un modèle léger coûte nettement moins cher qu'un modèle haut de gamme, à volume de tokens égal) et le nombre de **tokens** réellement échangés, en entrée comme en sortie. Une question simple posée à un modèle léger peut ainsi coûter une fraction de credit, alors qu'une session d'Agent mode qui modifie plusieurs fichiers en consommera bien davantage — d'où l'intérêt des réflexes de sobriété présentés dans les slides suivantes.

Source : *GitHub Docs*, « *Usage-based billing for individuals* » — docs.github.com/en/copilot/concepts/billing/usage-based-billing-for-individuals (consulté le 19/07/2026).

? Un « credit » à 0,01 \$ vous semble-t-il abstrait ou parlant ?

Plans individuels — credits inclus

PLAN	PRIX/MOIS	CREDITS DE BASE	FLEX	TOTAL
Copilot Pro	10 \$	1 000	500	1 500
Copilot Pro+	39 \$	3 900	3 100	7 000
Copilot Max	100 \$	10 000	10 000	20 000

Les credits « flex » varient selon les conditions de marché ; les credits « de base » sont fixes chaque mois.

Source : *GitHub Docs*, « *Usage-based billing for individuals* » — docs.github.com/en/copilot/concepts/billing/usage-based-billing-for-individuals (consulté le 19/07/2026).

Plans organisation – mécanique proche

- En Business/Enterprise, les credits sont souvent **mutualisés par utilisateur et par mois** plutôt qu'individuels, avec un budget d'équipe possible.
- Les administrateurs peuvent définir des **budgets de dépassement** (overage) au niveau organisation ou entreprise.
- Le principe reste identique : complétions gratuites, Chat/Agent facturés au token.

Source : GitHub Docs, « Budgets for usage-based billing » — docs.github.com/en/copilot/concepts/billing/budgets-for-usage-based-billing (consulté le 19/07/2026).

? Savez-vous quel plan Copilot est actif chez Transactis ?

Ce qui consomme un « request »

FONCTIONNALITÉ	COÛT EN REQUÊTES PREMIUM
Chat / CLI / Spaces	1 requête par prompt, multipliée par le taux du modèle
Revue de code automatique	13 requêtes par revue
Spark	4 requêtes par prompt (taux fixe)
Cloud agent	1 requête par session + 1 par commentaire de pilotage

Les actions autonomes d'un agent (appels d'outils internes) ne comptent pas comme des requêtes supplémentaires.

Source : GitHub Docs, « Copilot requests » — docs.github.com/en/copilot/concepts/billing/copilot-requests (consulté le 19/07/2026).

Tokens d'entrée vs tokens de sortie

Tokens d'entrée



coût de base

Tokens de sortie



jusqu'à 5x plus cher

- Les tokens de **sortie** (ce que Copilot écrit) coûtent nettement plus cher que les tokens d'**entrée** (ce que vous lui montrez).
- Demander « code seulement, sans explication » sur une tâche répétitive réduit le coût de façon significative.

Source : *GitHub Docs*, « Usage-based billing for individuals » ; *VS Code Blog*, « Improving token efficiency » (consultés le 19/07 et 08/07/2026).

Bonnes pratiques d'optimisation – 1/2

- **Ouvrez les fichiers pertinents**, fermez les fichiers inutiles dans l'éditeur.
- **Nettoyez l'historique** de conversation : supprimez les échanges inutiles, repartez à zéro sur un nouveau sujet.
- **Décomposez** une tâche complexe en plusieurs étapes simples plutôt qu'une seule demande énorme.
- **Formulez avec précision** : moins d'itérations = moins de tokens gaspillés.

Source : GitHub Docs, « Best practices for using GitHub Copilot » – docs.github.com/en/copilot/using-github-copilot/best-practices-for-using-github-copilot (consulté le 19/07/2026).

Bonnes pratiques d'optimisation – 2/2

- **Fournissez des exemples concrets** (entrée/sortie attendue) plutôt que de longues explications.
- **Restez sur le même modèle** dans une conversation : changer de modèle casse la mise en cache du contexte (donc plus cher).
- **Reformulez** si la réponse ne convient pas, au lieu d'empiler des questions correctrices.
- Donnez votre avis (👍 / 👎, accepter/refuser) : cela améliore la pertinence future, sans coût additionnel.

Source : GitHub Docs, « Best practices for using GitHub Copilot » – docs.github.com/en/copilot/using-github-copilot/best-practices-for-using-github-copilot (consulté le 19/07/2026).

? Laquelle de ces 8 pratiques allez-vous appliquer dès aujourd'hui sur TaskBoard ?

[EXERCICE ÉCLAIR](#)

Estimer un coût

1. Un modèle mini traite 2 000 tokens d'entrée + 500 de sortie. Un modèle haut de gamme peut coûter jusqu'à 20x plus par token.
2. Pourquoi une tâche répétitive (ex. renommer des variables) ne justifie-t-elle presque jamais le modèle le plus puissant ?

? Sur TaskBoard, quelle tâche de l'exercice mériterait un modèle puissant, et laquelle non ?

FICHE DE RÉFÉRENCE

Toutes les commandes

Variables, participants, slash commands, raccourcis, modèles.

CONTEXTE

#file #selection #codebase

```
#file:index.html  
/explain
```

- #file cible un fichier précis — le réflexe le plus rentable.
- #selection cible le texte sélectionné.
- #codebase cherche dans tout le projet — plus cher, à réserver aux questions transversales.

Source : *GitHub Docs*, « Chat cheat sheet » — docs.github.com/en/copilot/reference/chat-cheat-sheet (consulté le 19/07/2026).

Toutes les variables de contexte

VARIABLE	INCLUT
#file	Le fichier indiqué
#selection	Le texte sélectionné
#codebase	Une recherche dans tout le projet
#block	Le bloc de code courant
#class	La classe courante
#comment	Le commentaire courant
#function	La fonction/méthode courante
#line	La ligne courante
#path	Le chemin du fichier
#project	Le contexte du projet
#sym	Le symbole courant

Source : *GitHub Docs*, « Chat cheat sheet » — docs.github.com/en/copilot/reference/chat-cheat-sheet (consulté le 19/07/2026).

EXEMPLES REPRODUCTIBLES

Les autres variables # : exemples à coller — 1/2

Placez le curseur au bon endroit avant de coller chaque prompt — ces variables dépendent de la position de votre curseur, contrairement à `#file`.

Curseur dans la règle `.board` de `css/layout.css`, puis :
`#block` Explique ce que fait ce bloc de règles.

Curseur sur le sélecteur `.card` de `css/components.css`, puis :
`#class` Dans quelles pages ce sélecteur est-il utilisé ?

Curseur sur le commentaire `NOTE PEDAGOGIQUE` de `css/legacy-styles.css`, puis :
`#comment` Explique ce que ce commentaire demande de faire.

Curseur sur la ligne `--color-accent` de `css/variables.css`, puis :
`#line` Où cette variable est-elle réutilisée dans le projet ?

Les autres variables # : exemples à coller — 2/2

Ouvrez `about.html`, puis :
`#path` D'après ce chemin de fichier, à quoi sert cette page ?

N'importe quel fichier ouvert, puis :
`#project` Résume l'organisation de ce projet en 3 lignes.

Curseur sur le symbole `—color-accent` dans `css/variables.css`, puis :
`#sym` Liste les fichiers qui utilisent ce symbole.

`#function` cible une fonction ou méthode — moins pertinent sur un projet 100% HTML/CSS sans logique de code. Vous le retrouverez tel quel sur tout projet avec du JavaScript, du Python, etc.

Tous les participants de chat @

PARTICIPANT	DOMAINE
@workspace	Le code de votre espace de travail
@vscode	Commandes et fonctionnalités de VS Code
@terminal	Le terminal intégré
@github	Compétences GitHub (issues, PR)
@azure	Services Azure

Sur github.com : mentions génériques Discussions, Extensions, Files, Issues, Pull requests, Repositories.

Source : GitHub Docs, « Chat cheat sheet » — docs.github.com/en/copilot/reference/chat-cheat-sheet (consulté le 19/07/2026).

EXEMPLES REPRODUCTIBLES

Participants @ : exemples à coller — 1/2

Chaque participant oriente Copilot vers une source de connaissance précise. Voici 3 exemples que vous pourrez recoller tels quels cet après-midi sur TaskBoard.

```
@workspace Où est définie la classe TaskStore, et quels fichiers l'utilisent ?
```

```
@vscode Comment afficher le panneau des problèmes (Problems) dans cette fenêtre ?
```

```
@terminal Explique ce que fait la dernière commande que j'ai exécutée dans ce terminal.
```

```
@terminal ne fonctionne que si un terminal intégré est ouvert et a déjà exécuté au moins une commande.
```

? Lequel de ces 3 participants avez-vous déjà croisé sans savoir qu'il portait un nom précis ?

EXEMPLES REPRODUCTIBLES

Participants @ : exemples à coller — 2/2

@github et @azure demandent un contexte externe (un dépôt GitHub réel connecté, ou un abonnement Azure) : voici néanmoins deux exemples à garder sous la main pour le jour où vous en aurez besoin.

```
@github Résume les issues ouvertes sur ce dépôt qui mentionnent "priorité".
```

```
@azure Je veux héberger un petit site HTML/CSS statique, quelle est la première étape ?
```

Si vous n'avez pas ce contexte en salle, testez au moins @workspace et @vscode maintenant sur TaskBoard — ce sont les deux plus utiles au quotidien.

Commandes slash – VS Code

COMMANDE	ACTION
/explain	Explique le code de l'éditeur actif
/fix	Propose une correction
/fixTestFailure	Corrige un test qui échoue
/tests	Génère des tests unitaires
/new	Crée un nouveau projet
/clear	Nouvelle session
/help	Rappel des bases

Source : *GitHub Docs, « Chat cheat sheet »* – docs.github.com/en/copilot/reference/chat-cheat-sheet (consulté le 19/07/2026).

Commandes VS Code : exemples à coller — 1/2

Un exemple concret pour chaque commande, prêt à coller dans Copilot Chat sur TaskBoard.

```
#file:index.html  
/explain
```

```
#file:css/legacy-styles.css  
/fix  
Réécris ces règles avec les variables de css/variables.css, sans !important,  
en gardant EXACTEMENT le même rendu visuel.
```

```
#file:index.html  
/doc  
Ajoute un commentaire HTML au-dessus du formulaire, expliquant pourquoi  
il est volontairement statique.
```

```
/help
```


Commandes VS Code : exemples à coller — 2/2

/new

Crée un squelette de nouvelle page HTML pour TaskBoard, avec la même structure que index.html (en-tête, navigation, main, pied de page).

/clear

/tests et /fixTestFailure génèrent et corrigent des tests automatisés — utiles sur du code avec logique (JavaScript, Python...). Sur un projet 100% HTML/CSS comme TaskBoard, ils ne s'appliquent pas directement : c'est la checklist d'accessibilité (Étape 5 du guide d'exercice) qui joue ce rôle de vérification.

? Laquelle de ces 7 commandes n'avez-vous encore jamais utilisée ?

Commandes slash — Visual Studio & JetBrains

ÉDITEUR	COMMANDES PROPRES
Visual Studio	<code>/doc</code> (documenter un symbole) · <code>/optimize</code> (améliorer le temps d'exécution)
JetBrains	<code>/chronicle</code> (historique de session) · <code>/compact</code> (compresser le contexte CLI) · <code>/remote</code> (session CLI à distance)

Les deux éditeurs partagent aussi `/explain`, `/fix`, `/tests`, `/help`.

Source : *GitHub Docs*, « Chat cheat sheet » — docs.github.com/en/copilot/reference/chat-heat-sheet (consulté le 19/07/2026).

Visual Studio & JetBrains : exemples à coller

```
Sélectionnez la règle .card dans css/components.css, puis :  
/doc
```

```
Sélectionnez le sélecteur "div.card div.card-title.old" dans  
css/legacy-styles.css, puis :  
/optimize
```

```
/chronicle
```

Si vous êtes sur VS Code aujourd'hui : lisez ces exemples, ils fonctionnent à l'identique le jour où vous ouvrirez Visual Studio ou un IDE JetBrains.

Commandes slash — Xcode & github.com

ENVIRONNEMENT	COMMANDES PROPRES
Xcode	/doc · /simplify (simplifier la sélection) + communes
github.com	/new · /rename · /clear · /delete

Source : GitHub Docs, « Chat cheat sheet » — docs.github.com/en/copilot/reference/chat-cheat-sheet (consulté le 19/07/2026).



EXEMPLES REPRODUCTIBLES

Xcode & github.com : exemples à coller

Sélectionnez la section `<section class="board">` dans `index.html`, puis :
`/simplify`

Sur github.com, dans une conversation Copilot existante :

`/rename`
Exercice TaskBoard – 19 juillet

`/new`

`/clear` et `/delete` ne prennent pas de texte : ce sont des boutons/commandes d'action directe dans l'interface de conversation.

Copilot dans le terminal (CLI)

```
$ copilot  
> explique les règles du fichier css/legacy-styles.css
```

- Copilot CLI apporte le Chat directement dans le terminal, hors éditeur.
- Utile pour des tâches shell/scripts, ou quand aucun IDE n'est ouvert.
- Partage les commandes `/compact` et `/remote` vues côté JetBrains.

Source : *GitHub Docs, « Copilot agents concepts » — docs.github.com/en/copilot/concepts/agents (consulté le 19/07/2026).*

Compétences MCP sur github.com

COMPÉTENCE	ACTION
get_me	Vos infos utilisateur
search_users	Recherche d'utilisateurs
create_branch	Créer une branche
create_or_update_file	Ajouter/modifier un fichier
push_files	Pousser des fichiers
update_pull_request_branch	Mettre à jour une PR
merge_pull_request	Fusionner une PR

Source : *GitHub Docs, « Chat cheat sheet » — docs.github.com/en/copilot/reference/chat-cheat-sheet (consulté le 19/07/2026).*

Raccourcis clavier — toutes plateformes

ACTION	WIN/LINUX	MAC
Accepter	Tab	Option+Tab
Ignorer	Échap	Échap
Suivante	Alt+]	Option+]
Précédente	Alt+[	Option+[
Déclencher	Alt+\\	Option+\\
Panneau Copilot	Ctrl+Entrée	Ctrl+Retour

Source : *GitHub Docs*, « *Keyboard shortcuts for GitHub Copilot in the IDE* » — docs.github.com/en/copilot/reference/keyboard-shortcuts (consulté le 19/07/2026).

Tous les modèles disponibles

FOURNISSEUR	MODÈLES
OpenAI	GPT-5 mini · 5.3-Codex · 5.4 (+mini/nano) · 5.5 · 5.6 Luna/Sol/Terra
Anthropic	Haiku 4.5 · Sonnet 4.5/4.6/5 · Opus 4.5/4.6/4.7/4.8 · Fable 5
Google	Gemini 2.5 Pro · 3 Flash · 3.1 Pro · 3.5 Flash
Microsoft / autres	MAI-Code-1-Flash · Raptor mini · Kimi K2.7 Code

Source : *GitHub Docs*, « *Supported AI models in Copilot* » — docs.github.com/en/copilot/reference/ai-models/supported-models (consulté le 19/07/2026).

? Un modèle « mini » suffit-il pour expliquer `css/base.css` de TaskBoard ?

[CHOISIR SIMPLEMENT](#)

Changer de modèle, pas à pas

1. Ouvrez Copilot Chat.
2. Cliquez sur le nom du modèle en bas de la fenêtre.
3. Choisissez un autre modèle dans la liste.
4. Le nouveau modèle s'applique dès le prochain message.

Réflexe débutant : gardez le modèle par défaut pour vos premières questions.



[EXERCICE ÉCLAIR](#)

Comparer 2 modèles

```
#file:css/legacy-styles.css  
/explain  
Liste les problèmes de ce fichier CSS en 3 points maximum.
```

1. Collez ce prompt tel quel, une première fois avec un modèle « mini » sélectionné, puis une seconde fois dans une nouvelle conversation avec un modèle plus complet.
2. Comparez : longueur de la réponse, pertinence, temps perçu.

? Le modèle le plus puissant a-t-il vraiment donné une meilleure réponse pour cette tâche simple ?

Grille de décision : quel outil, pour quelle tâche ?

BESOIN	OUTIL RECOMMANDÉ
Comprendre un fichier	Chat + #file + /explain
Corriger un bug ciblé	Chat + #selection + /fix
Modifier plusieurs fichiers, sous contrôle	Agent mode
Tâche longue, déléguée	Cloud agent
Automatiser une tâche récurrente	Workflow agentique (on: daily)
Rédiger un mail ou un compte-rendu	Copilot Chat (M365), pas GitHub Copilot

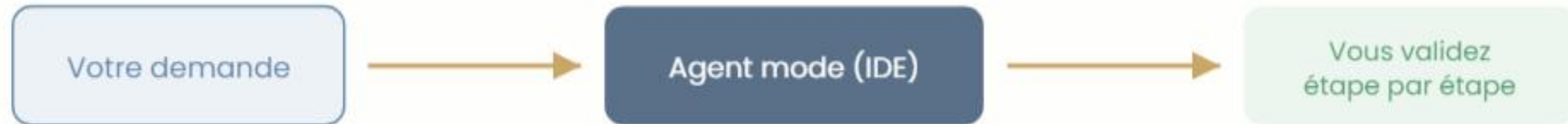
? Laquelle de ces 6 lignes correspond le mieux à votre prochaine tâche réelle ?

ÉCOSYSTÈME AVANCÉ

Agents, MCP, skills, hooks, plugins, workflows

Tout ce qui existe aujourd'hui, expliqué simplement.

Agent mode : un collègue à côté de vous



L'Agent mode transforme Copilot d'un simple assistant qui répond en un collaborateur qui **agit** : il peut lire plusieurs fichiers à la fois, proposer des modifications sur chacun d'eux, et exécuter des commandes dans le terminal. C'est le mode à privilégier dès qu'une tâche dépasse un seul fichier — par exemple, ajouter une nouvelle section à `index.html` et son style associé dans `css/components.css` en une seule demande.

La différence essentielle avec un simple Chat, c'est que vous restez **en direct, à côté de l'agent** : chaque modification proposée s'affiche avant d'être appliquée, vous pouvez interrompre à tout moment, et rien n'est écrit sur disque sans votre validation explicite.

Source : *GitHub Blog*, « *The difference between coding agent and agent mode* » (consulté le 08/07/2026).

Cloud agent : un collègue parti travailler seul

Le Cloud agent (parfois encore appelé « Coding agent ») pousse la délégation plus loin : au lieu de rester à côté de vous, il part travailler seul, de façon asynchrone, et revient plus tard avec un résultat complet à relire. Son fonctionnement suit toujours trois étapes explicites : il commence par une phase de **recherche** dans le dépôt pour comprendre le contexte, puis propose un **plan** d'implémentation avant d'écrire la moindre ligne, et enfin réalise les **modifications** sur une branche dédiée.

Une fois son travail terminé, c'est à vous de relire le résultat et de préparer la pull request — la revue humaine reste une étape obligatoire, jamais automatique. Le Cloud agent peut également s'appuyer sur des agents personnalisés, des automatisations planifiées (voir les workflows plus loin), et des connexions MCP pour élargir son champ d'action.

Source : *GitHub Docs*, « About the Copilot cloud agent » — docs.github.com/en/copilot/concepts/agents/cloud-agent (consulté le 19/07/2026).

? Confieriez-vous « ajouter un formulaire complet avec validation » à un agent qui travaille seul ?

*.agent.md : où les stocker ?

Un agent personnalisé peut être stocké à **deux niveaux**, et cette différence compte. Au niveau du **projet**, dans le dossier `.github/agents/` (ou `.claude/agents/` pour le format Claude) : l'agent est versionné avec le code, partagé avec toute l'équipe dès qu'il est commité, et ne s'applique qu'à ce dépôt. Au niveau **utilisateur**, dans `~/.copilot/agents/` (votre profil VS Code) : l'agent vous est personnel, disponible sur tous vos projets, mais invisible pour vos collègues.

Règle simple : un agent qui encode une convention d'équipe (comme notre `reviewer` pour TaskBoard) va dans le projet ; un agent qui reflète vos habitudes personnelles va dans votre profil utilisateur.

Source : VS Code Docs, « Custom agents » — code.visualstudio.com/docs/copilot/customization/custom-agents (consulté le 19/07/2026).

*.agent.md : créer l'agent, étape par étape

1. Dans la vue Chat, cliquez sur l'icône « Configure Chat », onglet **Agents**.
2. Choisissez « New Agent (Workspace) » pour un agent de projet (ou « New Agent (User) » pour votre profil).
3. Nommez le fichier — ce sera aussi le nom affiché dans la liste des agents.
4. Collez le contenu ci-dessous, propre à TaskBoard :

```
.github/agents/reviewer.agent.md

---
name: reviewer
description: Analyse uniquement le code de TaskBoard, ne modifie jamais rien
tools: [read, search]
---

Tu es un agent de revue de code pour TaskBoard. Tu ne dois JAMAIS
proposer de modification directe des fichiers. Ta sortie est toujours
structurée : constat, risque, recommandation.
```

Source : VS Code Docs, « Custom agents » — code.visualstudio.com/docs/copilot/customization/custom-agents (consulté le 19/07/2026).

[EXERCICE ÉCLAIR](#)

Tester l'agent reviewer

1. Ouvrez le menu déroulant des agents dans la fenêtre de chat, sélectionnez `reviewer`.
2. Collez le prompt ci-dessous et vérifiez qu'aucun fichier n'est modifié :

```
#file:index.html  
Analyse ce fichier et signale les risques éventuels.
```

? L'agent a-t-il respecté son périmètre déclaré (analyse seule, aucune modification) ?

Le mécanisme des sous-agents : handoffs

Un agent personnalisé peut **déléguer** à un autre agent grâce au champ `handoffs` de son en-tête : un bouton apparaît dans le chat, avec un texte au choix, qui bascule la conversation vers l'agent cible en lui transmettant un prompt suggéré. Un agent peut aussi lister, via le champ `agents`, les sous-agents qu'il a le droit d'invoquer lui-même. C'est exactement le principe évoqué plus tôt dans ce support : un agent « planificateur » restreint, qui transmet ensuite la main à un agent « relecteur » lui aussi restreint.

Source : VS Code Docs, « Custom agents » (champs `handoffs`, `agents`) — code.visualstudio.com/docs/copilot/customization/custom-agents (consulté le 19/07/2026).

SOUS-AGENTS

Exemple applicable : planner → reviewer sur TaskBoard

Créez un second agent `.github/agents/planner.agent.md`, qui ne produit jamais de code, seulement un plan — puis transmet la main à `reviewer` (créé juste avant) pour validation :

```
---
name: planner
description: Planifie une évolution de TaskBoard, sans écrire de code
tools: [read, search]
handoffs:
  - label: Envoyer au relecteur
    agent: reviewer
    prompt: Relis ce plan et signale les risques avant toute implémentation
    send: false
---
```

Tu proposes uniquement un plan d'implémentation en 3 à 5 étapes, jamais de code. Termine toujours en proposant l'envoi au relecteur.

Sélectionnez l'agent planner, puis collez :
Je veux ajouter un filtre par priorité sur le tableau TaskBoard. Propose un plan.

? Cliquez sur le bouton de handoff : le relecteur reçoit-il bien le plan complet ?

SKILLS

Skills : des compétences réutilisables

Une **skill** est un dossier autonome (`skills/nom-de-la-skill/SKILL.md`) qui décrit, en langage naturel, une compétence complète que Copilot peut mobiliser : quand l'utiliser, comment procéder étape par étape, et quels résultats produire. Contrairement à un simple prompt file, une skill peut embarquer des fichiers annexes au même endroit — gabarits, scripts, exemples — pour former un ensemble cohérent et directement réutilisable.

L'intérêt principal est la **capitalisation** : plutôt que de réexpliquer chaque fois « comment on fait une revue de code chez nous » dans le chat, on documente la compétence une seule fois, et toute l'équipe peut s'appuyer dessus de façon identique.

Source : GitHub Docs, « About plugins » — docs.github.com/en/copilot/concepts/agents/about-plugins (consulté le 19/07/2026).

SKILLS

Créer une skill pour TaskBoard, étape par étape

1. Dans `codebase_taskboard/`, créez le dossier `skills/validation-taskboard/`.
2. À l'intérieur, créez le fichier `SKILL.md` avec le contenu ci-dessous.
3. Enregistrez : aucune activation supplémentaire n'est nécessaire.

```
skills/validation-taskboard/SKILL.md
```

```
# Validation TaskBoard
```

Quand l'utiliser : avant d'ajouter un élément à une page (carte, champ de formulaire, badge), pour vérifier la cohérence avec le reste du projet.

Comment procéder :

1. Vérifier que toute couleur ou espacement utilisé provient de `css/variables.css` – jamais de valeur écrite en dur.
2. Vérifier que chaque nouveau champ de formulaire a un `<label>` associé (voir `docs/checklist-accessibilite.md`).
3. Rejeter toute proposition qui introduit du JavaScript : ce projet reste volontairement HTML/CSS uniquement.

Source : *GitHub Docs, « About plugins »* – docs.github.com/en/copilot/concepts/agents/about-plugins (consulté le 19/07/2026).

[EXERCICE ÉCLAIR](#)

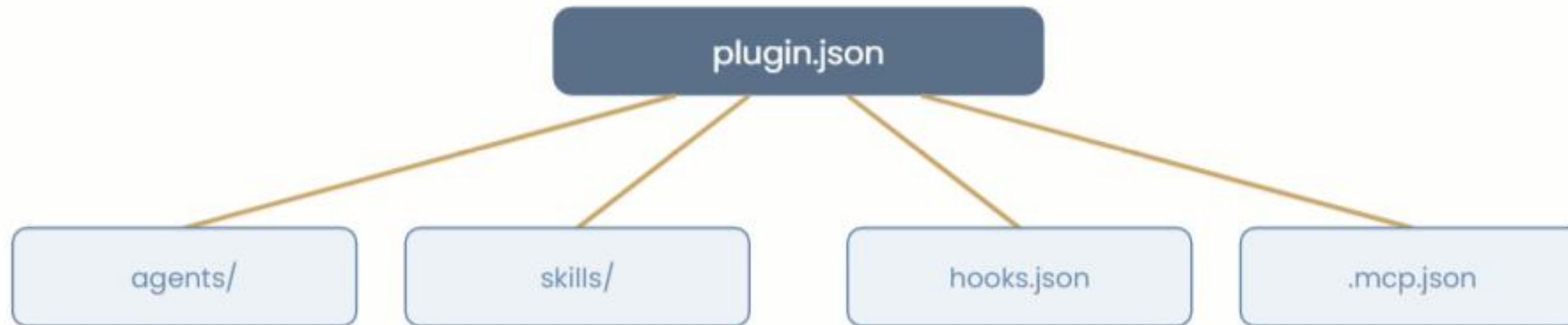
Tester la skill

1. Dans le chat, mentionnez le fichier de la skill pour la mettre en contexte, puis posez une demande qui devrait la déclencher :

```
#file:skills/validation-taskboard/SKILL.md  
Je veux ajouter un champ "tags" (liste de mots-clés) au formulaire de  
index.html. Applique la procédure de validation-taskboard avant de proposer du HTML/CSS.
```

? La réponse a-t-elle bien suivi les 3 étapes décrites dans la skill ?

Plugins : regrouper agents, skills, hooks, MCP



Un **plugin** est un paquet installable qui distribue, en une seule fois, un ensemble cohérent de bonnes pratiques Copilot à toute une équipe : des agents personnalisés, des skills, des hooks et des connexions MCP, réunis autour d'un fichier `plugin.json` qui décrit le tout. C'est la brique d'industrialisation la plus aboutie de l'écosystème : au lieu de copier des fichiers un par un entre projets, on installe un plugin entier.

Source : GitHub Docs, « About plugins » ; github.com/github/awesome-copilot (consultés le 19/07/2026).

PLUGINS

Installer un plugin, étape par étape

1. Ouvrez un terminal (ou Copilot CLI) à la racine de votre projet.
2. Parcourez le catalogue awesome-copilot pour repérer un plugin utile (ex. un plugin de revue de code standard).
3. Installez-le avec la commande ci-dessous, en remplaçant `<nom>` par le nom exact du plugin :

```
copilot plugin install <nom>@awesome-copilot
```

4. Vérifiez l'installation : les nouveaux agents, skills et prompt files du plugin apparaissent dans les menus correspondants de votre éditeur.
5. Testez un des composants installés sur TaskBoard, comme vous l'avez fait pour `reviewer`.

Source : *GitHub Docs, « About plugins » ; github.com/github/awesome-copilot (consultés le 19/07/2026).*

? Un plugin installé une fois est-il valable pour tous vos projets, ou seulement celui-ci ? À vérifier ensemble.

Hooks : automatiser des actions

Un **hook**, déclaré dans un fichier `hooks.json`, déclenche automatiquement une action à un moment précis du cycle de travail de Copilot — par exemple juste avant ou juste après une modification de fichier. C'est un mécanisme d'automatisation fine, qui fait partie des composants qu'un plugin peut regrouper aux côtés des agents et des skills, pour imposer par exemple une vérification systématique avant chaque changement.

Source : *GitHub Docs, « About plugins » — docs.github.com/en/copilot/concepts/agents/about-plugins (consulté le 19/07/2026).*

HOOKS

Créer un hook pour TaskBoard, étape par étape

1. À la racine de `codebase_taskboard/`, créez le fichier `hooks.json`.
2. Déclarez un hook qui rappelle de vérifier l'accessibilité après toute modification d'un fichier HTML :

```
hooks.json

{
  "hooks": [
    {
      "event": "afterEdit",
      "match": "*.html",
      "action": "notify",
      "message": "Page HTML modifiée – repassez docs/checklist-accessibilite.md avant de continuer."
    }
  ]
}
```

3. Enregistrez, puis modifiez n'importe quelle page HTML pour vérifier que le rappel apparaît.

Le format exact des hooks dépend de l'outil qui les exécute (plugin, CLI) — l'exemple ci-dessus illustre le principe : un événement, une condition, une action.

Source : *GitHub Docs*, « *About plugins* » — docs.github.com/en/copilot/concepts/agents/about-plugins (consulté le 19/07/2026).

MCP : connecter Copilot à vos outils



Le **Model Context Protocol (MCP)** est un standard ouvert qui définit comment Copilot peut se connecter à des outils et des données situés en dehors de votre code : une API interne, une base de données, un outil de gestion de projet. Plutôt que d'accorder un accès global, MCP fonctionne par **toolsets** — des regroupements de fonctionnalités que vous activez un par un, explicitement, ce qui permet de garder un contrôle fin sur ce que l'IA a le droit de faire.

Source : GitHub Docs, « Model Context Protocol (MCP) » — docs.github.com/en/copilot/concepts/context/mcp (consulté le 19/07/2026).

MCP : gouvernance et sécurité

Précisément parce qu'il ouvre Copilot vers l'extérieur, MCP est **désactivé par défaut** au niveau d'une organisation, via une politique dédiée (« MCP servers in Copilot ») réservée aux abonnements Business et Enterprise. Son activation se fait toolset par toolset, en suivant le principe du **moindre privilège** : on n'active que le strict nécessaire à la tâche, jamais « au cas où ».

Pour les dépôts protégés par GitHub Advanced Security, un filet de sécurité supplémentaire existe : la *push protection* bloque automatiquement les secrets détectés dans une réponse générée par l'IA, avant qu'ils ne soient publiés.

Source : GitHub Docs, « Model Context Protocol (MCP) » — docs.github.com/en/copilot/concepts/context/mcp (consulté le 19/07/2026).

? Qui, chez Transactis, devrait valider l'activation d'un nouveau serveur MCP ?

Exemple concret : le serveur MCP Context7

Context7 est un serveur MCP très utilisé qui résout un problème précis : un LLM (voir le module « Comprendre l'IA ») est entraîné sur une photo figée du monde à un instant donné — il connaît donc souvent une **ancienne version** d'une bibliothèque, et peut halluciner des fonctions qui n'existent plus ou pas encore. Context7 va chercher, au moment même de votre question, la documentation à jour de la bibliothèque concernée, directement depuis sa source, et la fournit à Copilot avant qu'il ne réponde.

Concrètement : au lieu de générer du code basé sur une version d'il y a un an, Copilot vérifie d'abord la documentation actuelle — utile par exemple si TaskBoard adoptait un jour un vrai framework CSS (Bootstrap, Tailwind CSS) plutôt que du CSS pur écrit à la main.

Source : github.com/upstash/context7 (consulté le 19/07/2026).

Installer Context7, étape par étape

1. Créez une clé API gratuite sur context7.com/dashboard (recommandé, augmente les limites d'usage).
2. À la racine de votre projet, créez le fichier `.vscode/mcp.json` s'il n'existe pas déjà.
3. Ajoutez la configuration ci-dessous, avec votre clé :

```
.vscode/mcp.json

{
  "servers": {
    "context7": {
      "type": "http",
      "url": "https://mcp.context7.com/mcp",
      "headers": { "CONTEXT7_API_KEY": "votre-clé-ici" }
    }
  }
}
```

4. Rechargez la fenêtre. Vérifiez la connexion via la Command Palette : « MCP: List Servers ».

Alternative rapide en ligne de commande, sans éditer de fichier à la main : `npx ctx7 setup` , puis suivez l'authentification proposée.

Source : github.com/upstash/context7 (consulté le 19/07/2026).

[⚙ EXERCICE ÉCLAIR](#)

Utiliser Context7

Quelles sont les classes utilitaires actuelles de Tailwind CSS pour créer une grille à 3 colonnes ? use context7

Le mot-clé `use context7` à la fin d'un prompt indique explicitement à Copilot d'aller chercher la documentation à jour avant de répondre.



OÙ TROUVER...

Trouver des MCP, skills et agents en ligne

RESSOURCE	CONTENU
github.com/mcp	Registre officiel : plus de 150 serveurs MCP, recherche, bouton d'installation
Extensions VS Code, recherche @mcp	Galerie de serveurs MCP installables directement depuis l'éditeur
github.com/github/awesome-copilot	Agents, skills, instructions et plugins communautaires, catégorisés par dossier
context7.com	Documentation à jour de milliers de bibliothèques, via le serveur MCP Context7

Source : github.com/mcp ; VS Code Docs, « MCP servers » — code.visualstudio.com/docs/copilot/customization/mcp-servers ; github.com/github/awesome-copilot (consultés le 19/07/2026).

? Parcourez github.com/mcp une minute : quel serveur pourrait servir à un projet Transactis réel ?

Agentic Workflows : automatiser en langage naturel

Les **workflows agentic** sont des automatisations de dépôt écrites en langage naturel plutôt qu'en script rigide : un fichier Markdown combine un en-tête YAML (qui définit le déclencheur et les permissions accordées) avec des instructions en français ou en anglais courant, que l'agent interprète à chaque exécution. Ce fichier est ensuite compilé en un fichier `.lock.yml` GitHub Actions « durci », puis exécuté comme n'importe quel workflow classique du dépôt.

Plusieurs moteurs peuvent exécuter ces workflows — `copilot`, mais aussi Claude, Codex ou Gemini selon le champ `engine` choisi — ce qui laisse la possibilité de comparer plusieurs IA sur la même automatisation.

```
-----  
on: daily  
permissions:  
  contents: read  
  issues: read  
  pull-requests: read  
engine: copilot  
-----  
Chaque jour, résume les issues ouvertes depuis 24h et poste un  
commentaire de synthèse sur l'issue de suivi #42.
```

Source : GitHub Docs, « About GitHub agentic workflows » — docs.github.com/en/copilot/concepts/agents/about-github-agentic-workflows (consulté le 19/07/2026).

WORKFLOWS

Ce qui est possible avec un workflow agentique

Rapports

Résumé quotidien de l'activité, synthèse des PR en attente, état d'avancement d'un projet.

Triage

Classer automatiquement les nouvelles issues par priorité ou par type.

Veille technique

Surveiller une dépendance et signaler une mise à jour importante.

Maintenance

Proposer une pull request de nettoyage régulier (dépendances, code mort).

La limite n'est pas technique mais organisationnelle : un workflow agentique a accès uniquement aux permissions explicitement listées dans son en-tête (`contents` , `issues` , `pull-requests` ...), suivant le même principe du moindre privilège que MCP.

Source : *GitHub Docs*, « *About GitHub agentic workflows* » — docs.github.com/en/copilot/concepts/agents/about-github-agentic-workflows (consulté le 19/07/2026).

WORKFLOWS

Créer un workflow, étape par étape

1. Dans `codebase_taskboard/`, créez le dossier `.github/workflows/` s'il n'existe pas.
2. Créez le fichier `weekly-todo-report.md` avec le contenu ci-dessous.
3. Commitez-le sur la branche par défaut : GitHub compile automatiquement un fichier `.lock.yml` associé.
4. Le workflow s'exécute ensuite seul, selon le déclencheur défini — ici chaque semaine.

```
.github/workflows/weekly-todo-report.md
```

```
-----  
on: weekly  
permissions:  
  contents: read  
engine: copilot  
-----
```

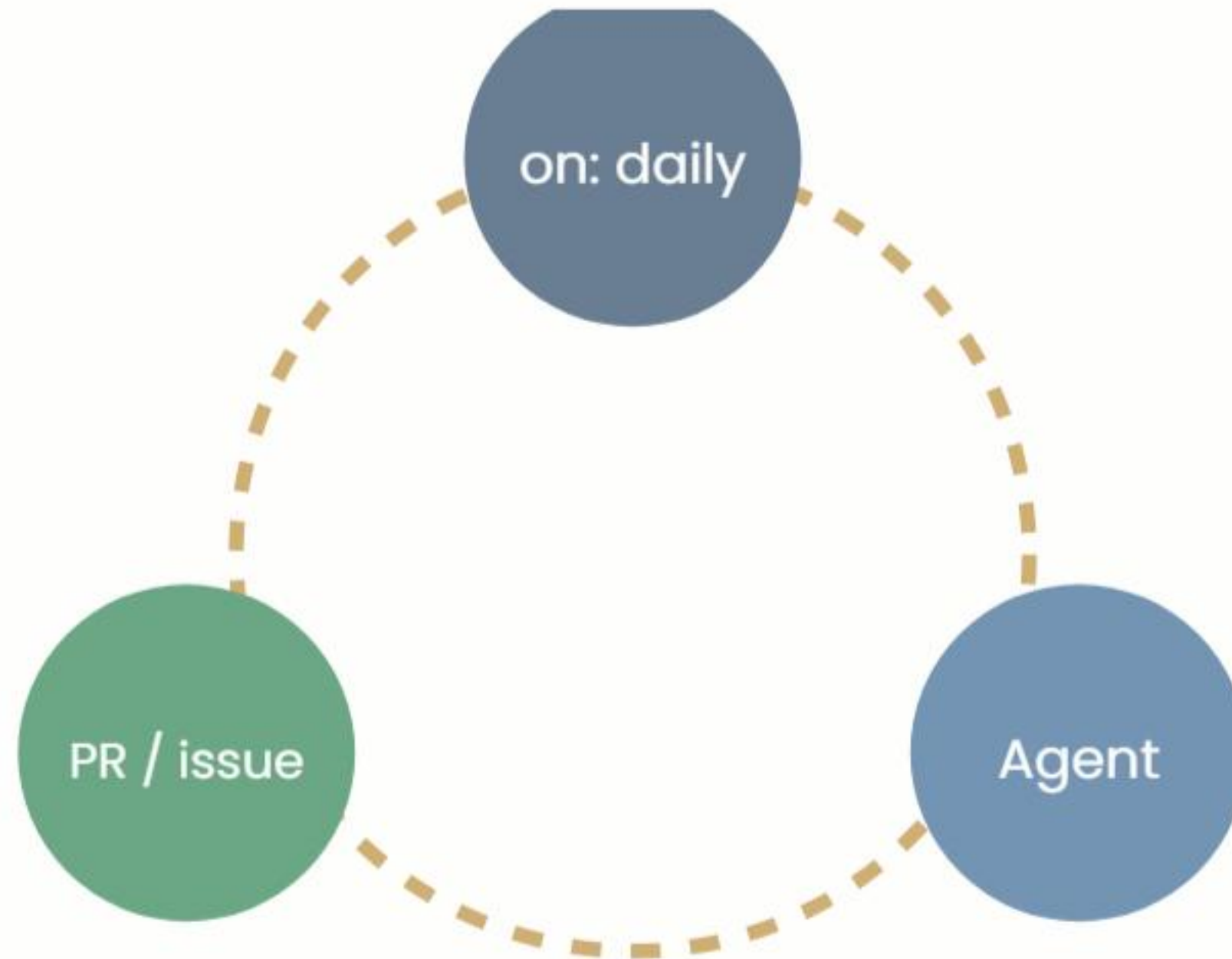
```
Chaque semaine, relis data/seed.json et le contenu des tâches  
TaskBoard restées au statut "todo" depuis plus de 7 jours, et  
rédige une note de relance résumant ces tâches.
```

Ce fichier peut aussi être écrit avec l'aide de Copilot lui-même : décrivez votre besoin en langage naturel, et demandez-lui de générer l'en-tête YAML correspondant.

Source : *GitHub Docs*, « *About GitHub agentic workflows* » — docs.github.com/en/copilot/concepts/agents/about-github-agentic-workflows (consulté le 19/07/2026).

LOOPS

Loops : exécutions récurrentes



Une « loop » n'est pas un composant technique séparé, mais une **façon d'utiliser** les workflows agentiques : en remplaçant le déclencheur manuel par `on: daily` (ou hebdomadaire, ou sur un événement précis comme l'ouverture d'une issue), on obtient une exécution qui se répète automatiquement, sans aucune intervention humaine à chaque cycle. L'agent se réveille, exécute sa mission, puis attend le prochain déclenchement.

awesome-copilot : le dépôt de référence

- Dépôt communautaire officiel GitHub, organisé en `agents/` , `instructions/` , `skills/` , `plugins/` , `cookbook/` , `hooks/` , `workflows/` .
- Fournit un `llms.txt` lisible par les IA, et un « Learning Hub » avec des guides.

```
copilot plugin install @awesome-copilot
```

Source : github.com/github/awesome-copilot (consulté le 19/07/2026).

? Quelle catégorie de ce dépôt exploreriez-vous en premier pour TaskBoard ?

Cookbook : des recettes prêtes à copier-coller

- Le dossier `cookbook/` d'awesome-copilot fournit des recettes concrètes pour travailler avec les API Copilot.
- Intérêt pour un débutant : partir d'un exemple qui fonctionne déjà, plutôt que d'une page blanche.

Source : github.com/github/awesome-copilot (consulté le 19/07/2026).

Extensions et agents tiers

- Le dossier `extensions/` référence des intégrations tierces avec l'écosystème Copilot.
- Il existe aussi des « third-party coding agents » documentés officiellement, à évaluer au cas par cas avant adoption en équipe.

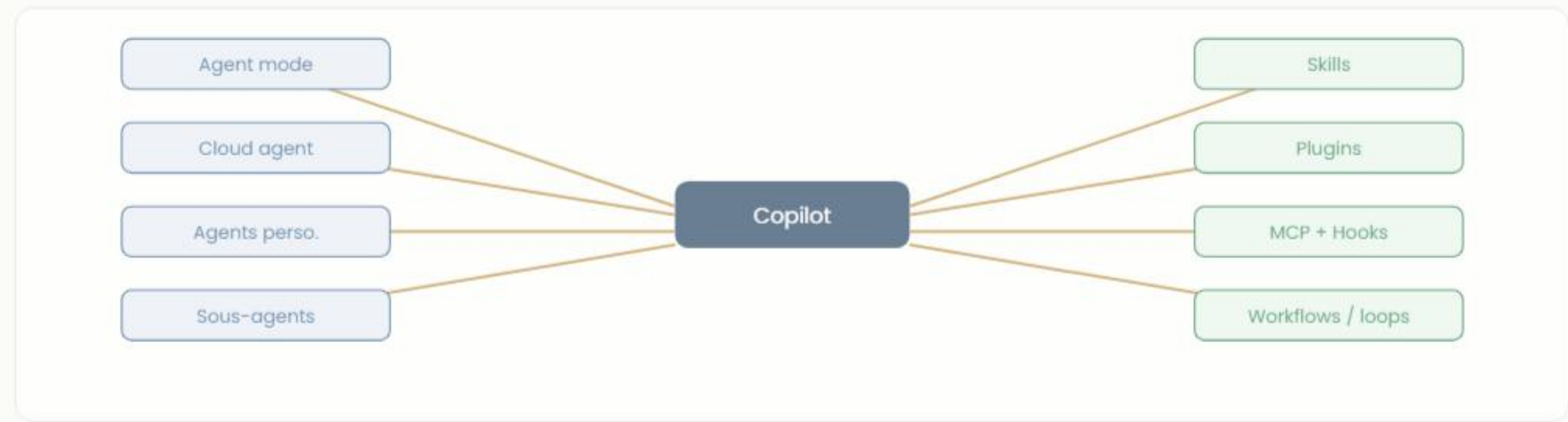
Source : github.com/github/awesome-copilot ; GitHub Docs, « About third-party coding agents » (consultés le 19/07/2026).

? Votre équipe a-t-elle déjà évalué un outil tiers connecté à Copilot ?



RÉCAP

Tout l'écosystème, en un schéma



? Sur ce schéma, quel élément avez-vous déjà utilisé, même sans le savoir ?

PERSONNALISATION

Configurer Copilot (doc officielle VS Code)

Instructions, fichiers, paramètres — la syntaxe exacte.

INSTRUCTIONS

copilot-instructions.md : à quoi ça sert

C'est le fichier d'instructions le plus simple et le plus puissant : toutes les règles qu'il contient sont lues **automatiquement** par Copilot, à chaque question posée sur le dépôt — dans le Chat, en Agent mode, et même par le Cloud agent. Vous n'avez rien à activer ni à mentionner : il suffit qu'il existe au bon endroit.

Emplacement unique : à la racine du projet, dans `.github/copilot-instructions.md`. Il n'existe pas de version « profil utilisateur » de ce fichier précis — pour une règle personnelle valable sur tous vos projets, voir `AGENTS.md` ou `CLAUDE.md` plus loin.

Source : VS Code Docs, « Custom Instructions » — code.visualstudio.com/docs/agent-customization/custom-instructions (consulté le 19/07/2026).

INSTRUCTIONS

copilot-instructions.md : créer le vôtre, étape par étape

1. À la racine de `codebase_taskboard/`, créez le dossier `.github/` s'il n'existe pas.
2. À l'intérieur, créez le fichier `copilot-instructions.md`.
3. Écrivez vos règles en Markdown libre — pas de frontmatter requis pour ce fichier.
4. Enregistrez. Posez n'importe quelle question sur le projet : le style de réponse doit déjà en tenir compte.

```
.github/copilot-instructions.md

# Copilot instructions – TaskBoard
## Conventions
- HTML5 sémantique, CSS pur, aucun JavaScript
- Nommage des classes CSS : kebab-case (ex. card-title, form-panel)
- Toutes les couleurs et espacements passent par css/variables.css
## Contraintes
- Ne jamais introduire de secret ou d'identifiant réel
## Style de réponse
- Code d'abord, explication courte ensuite
```

Source : VS Code Docs, « Custom instructions » — code.visualstudio.com/docs/agent-customization/custom-instructions (consulté le 19/07/2026).

INSTRUCTIONS CIBLÉES

*.instructions.md : à quoi ça sert

Contrairement à `copilot-instructions.md` (toujours actif partout), un fichier `*.instructions.md` ne s'applique **qu'aux fichiers qui correspondent à un motif précis**, grâce au champ `applyTo` de son en-tête. C'est le bon outil quand deux zones d'un même projet suivent des règles différentes — typiquement, un code ancien qu'on ne veut pas moderniser par erreur, face à un code récent qu'on veut faire évoluer librement.

Deux emplacements possibles : dans le projet, `.github/instructions/` (partagé avec l'équipe) ; ou dans votre profil utilisateur, `~/.copilot/instructions/` (personnel, valable sur tous vos projets).

Source : VS Code Docs, « Custom Instructions » — code.visualstudio.com/docs/agent-customization/custom-instructions (consulté le 19/07/2026).

INSTRUCTIONS CIBLÉES

*.instructions.md : créer le vôtre, étape par étape

1. Créez le dossier `.github/instructions/` dans `codebase_taskboard/`.
2. Créez le fichier `legacy-css.instructions.md`.
3. En première ligne, ouvrez un bloc `---`, ajoutez `applyTo` avec le motif de fichiers concernés, fermez par `---`.
4. Écrivez la consigne, puis enregistrez.

```
.github/instructions/legacy-css.instructions.md
```

```
-----  
applyTo: "css/legacy-styles.css"  
description: Conventions pour le CSS hérité  
-----
```

Ce fichier contient un ancien CSS. Ne proposez que des refactors qui gardent EXACTEMENT le même rendu visuel. N'ajoutez jamais de JavaScript, même pour "simplifier" une interaction.

`applyTo: "**"` applique l'instruction à tout le dépôt — utile pour tester rapidement le mécanisme avant de le restreindre.

Source : VS Code Docs, « Custom instructions » — code.visualstudio.com/docs/agent-customization/custom-instructions (consulté le 19/07/2026).

INSTRUCTIONS

AGENTS.md : pourquoi, où, comment

Pourquoi ce fichier plutôt que `copilot-instructions.md` ? `AGENTS.md` est un format ouvert, de plus en plus adopté par différents outils IA au-delà de Copilot — un même fichier peut donc servir à plusieurs assistants si votre équipe en utilise plusieurs. Sa différence essentielle avec `copilot-instructions.md` : il peut être **imbriqué**, c'est-à-dire présent aussi dans des sous-dossiers, chacun affinant les règles de sa propre zone (nécessite le paramètre `chat.useNestedAgentsMdFiles`).

1. Créez `AGENTS.md` à la racine de `codebase_taskboard/` avec les règles générales du projet.
2. Optionnel : créez un second `AGENTS.md` dans `css/` , avec une règle propre à ce dossier.

`css/AGENTS.md`

Ce dossier contient tous les styles du projet. `legacy-styles.css` est un code hérité, en attente de nettoyage. Toute proposition doit garder EXACTEMENT le même rendu visuel — aucun JavaScript, jamais.

Source : VS Code Docs, « Custom instructions » — code.visualstudio.com/docs/agent-customization/custom-instructions (consulté le 19/07/2026).

INSTRUCTIONS

CLAUDE.md : pourquoi, où, comment

Pourquoi ce fichier ? Si votre équipe utilise déjà Claude Code ou un autre outil compatible avec ce standard en plus de Copilot, `CLAUDE.md` évite de dupliquer les mêmes règles dans deux fichiers différents — un seul fichier sert à tous les outils compatibles. Sa différence avec `*.instructions.md` : il n'a pas de ciblage par `applyTo`, il s'applique globalement au dépôt (ou à vous, en version personnelle).

EMPLACEMENT	PORTÉE
Racine du projet, <code>CLAUDE.md</code>	Partagé avec toute l'équipe sur ce dépôt
<code>.claude/CLAUDE.md</code>	Partagé avec toute l'équipe, rangé à part
<code>~/ .claude/CLAUDE.md</code>	Personnel, valable sur tous vos projets

Source : VS Code Docs, « Custom instructions » — code.visualstudio.com/docs/agent-customization/custom-instructions (consulté le 19/07/2026).

? Votre équipe utilise-t-elle déjà, ou envisage-t-elle, un second outil IA en plus de Copilot ?

PROMPT FILES

*.prompt.md : créer un prompt réutilisable

Un prompt file encapsule une demande complète et réutilisable, invocable d'un simple `/nom-du-prompt` — utile pour une tâche que votre équipe répète souvent (revue de code standard, gabarit de nouveau composant).

1. Dans la vue Chat, icône ⚙️ « Configure Chat » → « New Prompt (Workspace) » (projet) ou « New Prompt (User) » (personnel).
2. Nommez le fichier — ce nom devient la commande slash.
3. Complétez le frontmatter et le corps du prompt.

```
.github/prompts/code-review.prompt.md
```

```
---
```

```
description: Revue de code standard TaskBoard
```

```
agent: ask
```

```
tools: [read, search]
```

```
---
```

```
Analyse le diff fourni. Vérifie : conventions de nommage, absence de secrets, couverture de tests. Restitue une checklist avec un niveau de risque (faible / moyen / élevé).
```

Emplacement projet : `.github/prompts/` . Emplacement personnel : dossier de profil VS Code, via le même menu. Invocation : tapez `/code-review` dans le chat.

Source : VS Code Docs, « Prompt files » — code.visualstudio.com/docs/copilot/customization/prompt-files (consulté le 19/07/2026).

Les paramètres à connaître, prêts à coller

Ces réglages se collent dans `settings.json` (Command Palette → « Preferences: Open User Settings (JSON) »), ou pour un réglage de projet, dans `.vscode/settings.json`.

```
{
  "chat.useAgentsMdFile": true,
  "chat.useNestedAgentsMdFiles": true,
  "chat.useClaudeMdFile": true,
  "chat.includeApplyingInstructions": true,
  "chat.instructionsFilesLocations": [".github/instructions"],
  "github.copilot.chat.organizationInstructions.enabled": true
}
```

PARAMÈTRE	RÔLE
<code>chat.instructionsFilesLocations</code>	Emplacements des dossiers d'instructions
<code>chat.useAgentsMdFile</code>	Active/désactive le support AGENTS.md
<code>chat.useNestedAgentsMdFiles</code>	Autorise des AGENTS.md par sous-dossier
<code>chat.useClaudeMdFile</code>	Active/désactive le support CLAUDE.md
<code>chat.includeApplyingInstructions</code>	Active les instructions ciblées par motif
<code>github.copilot.chat.organizationInstructions.enabled</code>	Active les instructions au niveau organisation

Source : VS Code Docs, « Custom Instructions » — code.visualstudio.com/docs/agent-customization/custom-instructions (consulté le 19/07/2026).

GÉNÉRER AUTOMATIQUEMENT

3 commandes pour générer vos instructions

COMMANDE	EFFET
/init	Analyse le projet et génère des instructions de base
/create-instruction	Génère un fichier d'instructions ciblé, avec l'aide de l'IA
/instructions	Ouvre le menu de configuration des instructions

```
/init
```

Source : VS Code Docs, « Custom Instructions » — code.visualstudio.com/docs/agent-customization/custom-instructions (consulté le 19/07/2026).

? Testeriez-vous /init directement sur TaskBoard cet après-midi ?

Les couches d'instructions, empilées

copilot-instructions.md — toujours actif, tout le dépôt

*.instructions.md + applyTo — actif si le fichier correspond

AGENTS.md — imbriqué, le plus proche du fichier gagne du détail

CLAUDE.md — global au dépôt, multi-outils

Ces 4 couches ne s'excluent pas : elles se **combinent**. Si votre question porte sur `css/legacy-styles.css`, et que ce fichier est couvert à la fois par `copilot-instructions.md`, un `*.instructions.md` ciblé, et un `AGENTS.md` local à `css/`, Copilot lit les trois et en tient compte ensemble — il n'y a pas de « gagnant unique » qui écraserait les autres, contrairement à ce qu'on pourrait attendre d'un système de priorité classique.

Bonne pratique : gardez `copilot-instructions.md` court et général ; réservez les règles très spécifiques aux fichiers ciblés (`*.instructions.md`, `AGENTS.md` imbriqué), pour éviter les contradictions entre couches.

Source : VS Code Docs, « Custom instructions » — code.visualstudio.com/docs/agent-customization/custom-instructions (consulté le 19/07/2026).

Instructions au niveau organisation

Au-dessus du dépôt, une organisation comme Transactis peut définir des instructions communes à **tous ses dépôts**, sans dépendre de chaque équipe pour les dupliquer partout. Cette couche s'active via le paramètre `github.copilot.chat.organizationInstructions.enabled`, et vient s'ajouter — pas remplacer — aux instructions de chaque projet individuel.

Source : VS Code Docs, « Custom instructions » — code.visualstudio.com/docs/agent-customization/custom-instructions (consulté le 19/07/2026).

? Quelle règle, commune à tous les projets Transactis, mériterait d'être définie une seule fois au niveau organisation ?

PROMPT ENGINEERING

Bien formuler sa demande

La compétence la plus rentable — écrire à Copilot, et écrire à Copilot Chat.

Écrire à Copilot ≠ écrire à Copilot Chat

Il existe en réalité **deux façons distinctes** de « parler » à Copilot, et un développeur doit maîtriser les deux. La première est **l'écriture inline** : pendant que vous codez, Copilot lit le fichier ouvert (le code déjà présent, les commentaires, les noms de classes) comme un prompt implicite, et propose une suggestion en gris, sans aucune fenêtre de dialogue. La seconde est **Copilot Chat** : une vraie conversation, où vous rédigez une demande explicite, avec du texte, des commandes (`/explain` , `/fix` ...) et des références de contexte (`#file` ...).

	ÉCRITURE INLINE	COPILOT CHAT
Où	Directement dans le fichier de code	Fenêtre de conversation séparée
Le prompt	Le code et les commentaires déjà écrits	Une phrase ou une question explicite
La réponse	Une suggestion à accepter (Tab) ou refuser (Échap)	Une explication, un extrait, un dialogue possible
Bon réflexe	Un commentaire clair juste avant le code à générer	Les 4 ingrédients (slide suivante)

Source : GitHub Docs, « What is GitHub Copilot? » — docs.github.com/en/copilot/get-started/what-is-github-copilot (consulté le 19/07/2026).

? Aviez-vous déjà remarqué que vous « parliez » à Copilot même sans ouvrir le Chat ?

Bien écrire un prompt inline, dans le code

Un commentaire précis, placé juste avant l'endroit où vous voulez du code, agit comme un prompt à part entière — c'est la technique la plus utilisée par les développeurs au quotidien, bien avant d'ouvrir le Chat.

```
←!— Carte de tâche : titre, badge de priorité en haut à droite,  
description sur 2 lignes maximum, date d'échéance en bas →
```

1. Collez ce commentaire dans `index.html`, juste avant une balise `<article class="card">` vide.
2. Appuyez sur Entrée, patientez une seconde : une suggestion grise doit apparaître.
3. `Tab` pour accepter, `Échap` pour refuser, `Alt+]` pour voir une suggestion alternative.

Plus le commentaire est précis (structure, contraintes), plus la suggestion inline est directement utilisable — exactement le même principe que pour Copilot Chat.

Source : *GitHub Docs*, « *Best practices for using GitHub Copilot* » — docs.github.com/en/copilot/using-github-copilot/best-practices-for-using-github-copilot (consulté le 19/07/2026).

Recette d'une bonne demande à Copilot Chat

- **Contexte** — #file
- **Objectif** — précis, mesurable
- **Contrainte** — style, comportement à garder
- **Vérification** — demandez ce qui reste incertain

EXEMPLE

Avant / après, dans Copilot Chat

✗ **Flou**

« Améliore mon code. »

✓ **Précis**

« #file:css/legacy-styles.css — Réécris avec les variables de css/variables.css, sans !important. Garde EXACTEMENT le même rendu visuel. »



9 principes de prompt engineering

#	PRINCIPE
1	Du général au précis
2	Donnez des exemples
3	Découpez les tâches complexes
4	Éliminez le flou
5	Précisez les bibliothèques
6	Montrez le code pertinent
7	Itérez plutôt qu'abandonner
8	Gardez une conversation propre
9	Un bon code de base aide Copilot

Source : GitHub Docs, « Prompt engineering for GitHub Copilot » — docs.github.com/en/copilot/concepts/prompting/prompt-engineering (consulté le 19/07/2026).

EXERCICE ÉCLAIR

Une demande complète, sur task-detail.html

Appliquez les 4 ingrédients vous-même, sur un fichier que vous n'avez pas encore ciblé aujourd'hui.

```
#file:task-detail.html  
Objectif : ajoute une section "Historique" sous les commentaires, avec  
2 entrées statiques (date + action).  
Contrainte : réutilise les classes déjà définies dans css/components.css,  
n'en crée aucune nouvelle.  
Vérification : signale si une classe existante ne convient pas exactement.
```

? Repérez les 4 ingrédients dans ce prompt : lequel a le plus influencé la réponse ?

EXERCICE ÉCLAIR

Transformez une demande floue

```
#file:css/legacy-styles.css  
améliore ce fichier
```

1. Collez d'abord ce prompt flou tel quel dans Copilot Chat et observez la réponse.
2. Réécrivez-le avec les 4 ingrédients (contexte déjà donné par `#file`, objectif précis, contrainte, vérification), collez la nouvelle version, et comparez les deux réponses.

? Qui partage sa version, et la différence observée entre les deux réponses ?

SÉCURITÉ

Les règles à connaître

3 RÈGLES

Les 3 règles d'or

- **1.** Vous restez responsable du code produit.
- **2.** Aucune donnée sensible soumise à l'IA.
- **3.** Toute proposition est relue par un humain.



CAS CONCRET

Ce qu'il ne faut jamais coller



Mot de passe, clé d'API, donnée client réelle.



Code sans secret, données fictives comme TaskBoard.



RÔLES

Qui est responsable de quoi ?

- **Vous** — formulez, relisez, décidez.
- **Le référent technique** — valide avant mise en production.
- **L'équipe sécurité** — définit les règles, reste le contact en cas de doute.

Copilot n'apparaît jamais dans cette liste : il assiste, il ne décide pas.

⚙ EXERCICE ÉCLAIR

Checklist sécurité

1. Notez 3 données que vous ne soumettez jamais à Copilot.



13H15-17H30

Place à la pratique : TaskBoard

10 étapes, 3 niveaux, un vrai projet HTML/CSS — sans une ligne de JavaScript.

LE PROJET

TaskBoard, ce que vous allez manipuler

- Un site HTML/CSS uniquement, sans build, sans installation — accessible même sans aucune notion de programmation.
- 12 fichiers : 3 pages HTML, 6 fichiers CSS dont un à améliorer, une documentation.

Ouvrez `codebase_taskboard/index.html` dans un navigateur pour démarrer.

CAS CONCRETS

3 usages typiques sur TaskBoard

Comprendre

`#file:index.html` `/explain` — avant de toucher à quoi que ce soit.

Améliorer

`#file:css/legacy-styles.css` `/fix` — rendre propre sans changer le rendu.

Documenter

`#selection` `/doc` — préparer une revue humaine.



Les fichiers du projet

```
index.html · task-detail.html · about.html  
css/variables.css · base.css · layout.css · components.css · responsive.css  
css/legacy-styles.css  ← à améliorer  
docs/architecture.md  
docs/checklist-accessibilite.md  
README.md
```

Aucun fichier JavaScript dans ce projet — tout se passe en HTML et en CSS.

3 NIVEAUX

Comment fonctionne l'exercice

● Facile

● Intermédiaire

● Pro

- ● **Facile** — prompts prêts à recopier.
- ● **Intermédiaire** — prompts à adapter.
- ● **Pro** — périmètre libre, plugins et workflows inclus.

◀

▶

APERÇU

Les 10 étapes de l'exercice

- 1. Explorer · 2. Cibler avec #file/#selection · 3. Instructions de dépôt · 4. Améliorer le legacy · 5. Tests · 6. Documentation · 7. Agent mode · 8. Prompt file · 9. Agent personnalisé · 10. MCP/hooks/workflow (bonus pro)

? Par quelle étape allez-vous commencer ?



[⚡ MINI-QUIZ ÉCLAIR](#)

Écosystème avancé — 3 questions

1. Quelle est la différence entre Agent mode et Cloud agent ?
2. Dans quel fichier place-t-on les instructions toujours actives pour un dépôt ?
3. MCP est-il activé par défaut au niveau organisation ?



SYNTHÈSE

« **Ce qu'il faut retenir**

»

Erreurs fréquentes

- Accepter sans lire. Question floue. Donnée sensible « juste cette fois ». `#codebase` par facilité. MCP activé sans gouvernance.

Glossaire — 1/2

AI Credit

Unité de facturation, 1 credit = 0,01 \$.

Toolset

Regroupement de fonctionnalités MCP activées explicitement.

Agent mode

Agent en direct, dans l'éditeur, supervisé.

Cloud agent

Agent autonome : recherche, plan, code, PR.



Glossaire — 2/2

Skill

Compétence réutilisable (`SKILL.md`).

Plugin

Paquet regroupant agents, skills, hooks, MCP.

applyTo

Motif de fichiers ciblés par une instruction.

Workflow agentique

Automatisation en langage naturel, déclenchée par `on:` .



Sources GitHub Docs

PAGE	URL (DOCS.GITHUB.COM)
What is GitHub Copilot?	/en/copilot/get-started/what-is-github-copilot
Chat cheat sheet	/en/copilot/reference/chat-cheat-sheet
Keyboard shortcuts	/en/copilot/reference/keyboard-shortcuts
Supported AI models	/en/copilot/reference/ai-models/supported-models
Prompt engineering	/en/copilot/concepts/prompting/prompt-engineering
Best practices	/en/copilot/using-github-copilot/best-practices-for-using-github-copilot

Sources GitHub Docs (suite)

PAGE	URL (DOCS.GITHUB.COM)
Usage-based billing (individuels)	/en/copilot/concepts/billing/usage-based-billing-for-individuals
Budgets for usage-based billing	/en/copilot/concepts/billing/budgets-for-usage-based-billing
Copilot requests	/en/copilot/concepts/billing/copilot-requests
About the Copilot cloud agent	/en/copilot/concepts/agents/cloud-agent
About plugins	/en/copilot/concepts/agents/about-plugins
Model Context Protocol (MCP)	/en/copilot/concepts/context/mcp
About GitHub agentic workflows	/en/copilot/concepts/agents/about-github-agentic-workflows

Sources VS Code Docs & awesome-copilot

SOURCE	URL
VS Code Docs, Custom instructions	code.visualstudio.com/docs/agent-customization/custom-instructions
awesome-copilot (dépôt communautaire)	github.com/github/awesome-copilot
Mastering GitHub Copilot for Paired Programming	github.com/microsoft/Mastering-GitHub-Copilot-for-Paired-Programming
Microsoft Learn, Decide which Copilot	learn.microsoft.com

Toutes ces pages ont été consultées le 19/07/2026. En cas de doute, la page officielle a toujours raison sur ce support.

⚙ EXERCICE ÉCLAIR

Votre plan d'action

1. Un usage concret cette semaine.
2. Une règle de sécurité.
3. Un réflexe d'optimisation de tokens.



POINTS CLÉS

Ce qu'il faut retenir

- 01 1 AI Credit = 0,01 \$; les complétions sont gratuites, le Chat/Agent coûte.
- 02 Tokens de sortie ≈ 5x plus chers que les tokens d'entrée.
- 03 Agent mode = supervisé ; Cloud agent = autonome (recherche/plan/code).
- 04 Agents, skills, hooks, MCP se regroupent dans des plugins.
- 05 copilot-instructions.md + *.instructions.md + AGENTS.md se combinent.
- 06 Les workflows agentiques automatisent en langage naturel, y compris en boucle.



Merci.

Des questions ? Contactez-moi directement.

hello@dhcompany.pro

dhcompany.pro

ia4africa.org

@DigitalMaker237 sur LinkedIn & X